Course 3 — The Living Apparatus

Building a beis medrash with memory

> A third self-study curriculum for the owner of this system and the students working on it. **Course 1** taught the pipeline: how a question becomes a grounded, cited answer, and how the capture loop makes the corpus yours. **Course 2** taught the intelligence layer: expand → retrieve → fuse → reason → verify. **This course teaches the phase we are about to build — Phase 7, "The Living Apparatus"** (BUILD_PLAN_PHASE7.md): the transformation of a trustworthy lookup system into a *memory* — typed, compounding, self-questioning, and able to convert better models into better knowledge with no code changes.
>
> These are class notes written **at the start of the semester**. Most of the code these lessons describe does not exist yet; the plan's Prompts 33–42 will build it, one at a time, largely by smaller models working under the harness you will learn to supervise here. That is not a weakness of the course — it is the point. You are not being trained to *use* this phase. You are being trained to *build and judge* it.

Why this course exists

After Courses 1 and 2 you own a system that is genuinely rare: it answers only from sources, cites every claim, refuses honestly, and lets a scholar's correction become retrievable knowledge behind a gate. And yet the honest assessment that opens the Phase 7 plan stands: **knowledge accumulates in it as a pile, not a structure.** A connection is a paragraph with two references stapled to it. A term is a chunk. Nothing composes. The system never asks a question. An approved correction improves retrieval once and is never heard from again.

Phase 7 closes three gaps — flat knowledge, a capped harness, and squandered expert attention — and this course exists so that every person working on the project understands, from first principles, *why* each piece is shaped the way it is. A student who merely implements Prompt 35 can get it to pass its tests. A student who understands Module 3 of this course will notice when an implementation quietly lets the graph rescue a refusal — the one bug the tests are least likely to catch and the trust core can least afford.

How to use this

- **Prerequisites:** Courses 1 and 2, or equivalent comfort with this repo — you should already be able to explain hybrid retrieval, the default-deny gate, RRF fusion, and dependency injection without notes.
- **Three things per lesson, as always:** the *idea* (first principles), the *anchor* (where it lives — in existing code, or in the plan's prompt that will build it), and a *hands-on*. Because this phase is under construction, some hands-ons run today and some are marked **(after Prompt NN)** — do those as the prompts land; they double as acceptance checks.
- **Pace:** one module per sitting. Modules 0–2 are the foundations (structure and free edges); 3–5 are the working systems (walking retrieval, the inquiry queue, the agent); 6–8 are compounding and measurement; Module 9 is your job description.
- **The invariants never move.** Every lesson that touches retrieval or writing restates the same three: default-deny fires before any model call; printed text is immutable; nothing model-drafted is indexed without a gate. If a lesson ever seems to bend one of these, you have misread the lesson. Reread it.

What you'll be able to do by the end

Explain why similarity search cannot represent a machlokes and what a typed edge adds; classify any assertion in the system by its evidence code and say what that code licenses; bootstrap thousands of edges with zero model calls and say why they are *more* trustworthy, not less; reason about graph-expanded retrieval without endangering the refusal gate; design a question the system should ask the Rav, and say why it beats the question below it in the queue; read a chavrusa agent trace like a teacher reads a talmid's notebook; explain what makes a harness *elastic* and how we will measure it; and review a pull request from a smaller model with the specific eye this project's trust core demands.

Module 0 — From a pile to a structure

Goal: see precisely what the system knows today, why that form of knowing cannot compound, and hold the whole Phase 7 map before we open any single box.

Lesson 0.1 — What the system actually knows

> Module 0, Lesson 1 · ~10 min + hands-on. The one question this answers: **when we say maayan "knows" something, what — exactly — is stored, and what can be done with it?**

Strip away every layer of the system and one data shape remains: the **Chunk** (`maayan/corpus/models.py`). A chunk is a passage of text with an identity (`id`, derived deterministically from `source|ref|lang`), an address (`ref`, `book`, `section_path`), a language, a provenance tag (`source`: `sefaria`, `chabad`, `expert`, `derived`, `term`, `shiur`), and free-form `metadata`. Everything the system knows — the Alter Rebbe's words, the Rav's connection, a model-drafted and gate-approved definition — is one of these.

Knowing this, you can state exactly what the system can *do* with its knowledge:

1. **Embed it** — turn the text into vectors (dense + sparse) so that
2. **similar text can find it** — hybrid search returns the chunks nearest a query, and
3. **filter it** — by `source`, by `book`, because those are fields.

That is the complete list. Notice what is *not* on it: the system cannot ask "what else speaks about this passage?", cannot follow a reference, cannot notice that two of its own chunks contradict each other, and cannot combine two pieces of knowledge into a third. Each chunk is an island that a query's embedding may or may not wash up on.

This is not a criticism of the design so far — chunks-with-provenance is the *correct* foundation, and the trust core (gates, refusal, retraction) is built on it and survives Phase 7 untouched. But it is a **pile**: it grows by accretion, and item number 4,270 makes items 1 through 4,269 no more useful than they were.

Anchor: `maayan/corpus/models.py` (the `Chunk` model and `chunk_id`); `maayan stats` for the census.

Hands-on: run `uv run maayan stats` and read the counts by source. Then run `uv run maayan search "אהבה בתענוגים" --source expert --k 3` and look at one expert connection. Ask yourself: if you wanted to know *every* expert connection touching Tanya chapter 18 — not the ones

similar to some query, all of them — what would you run? Sit with the fact that there is no good answer. That absence is Phase 7's reason to exist.

Lesson 0.2 — Similarity is not structure

> Module 0, Lesson 2 · ~12 min. The one question this answers: **why can't better embeddings fix this? What can nearest-neighbor search never represent, no matter how good the model?**

It is tempting to think the pile problem is a retrieval-quality problem — that a stronger embedder, or the Course 2 expansion tricks, would surface everything relevant and the structure question would dissolve. It would not, and the reason is worth owning from first principles.

An embedding maps a text to a point in space such that *related texts land near each other*. Nearness is one number. But the relationships that carry the meaning of a corpus like ours are **typed and directional**: this ma'amar *quotes* that pasuk; this chapter *explains* that concept; this Torah Ohr discourse *is the mashal for* what that Tanya chapter states flatly; Reb Hillel *argues with* the Tzemach Tzedek's reading. Two of those relationships can hold between the same pair of texts as a third fails to — and "0.83 cosine similarity" says nothing about which.

Three concrete things similarity cannot do:

- **It cannot distinguish agreement from dispute.** Two passages arguing about בירורים embed *closer together* than two passages that agree about unrelated things. The machlokes — often the most valuable thing a scholar knows about a pair of sources — is invisible; worse, it is *positively confused* with affinity.
- **It cannot compose.** From "A relates to B" and "B relates to C," similarity gives you nothing about A and C. A typed edge does: *A quotes pasuk P, C quotes pasuk P* is a real, checkable relationship between A and C — one that Lesson 2.2 will show we can compute for free.
- **It cannot be audited.** When a search result is wrong, there is nothing to retract — the geometry is what it is. When a typed edge is wrong, a named person retracted a specific assertion for a stated reason. Structure is what makes knowledge *governable*, and governance (gates, provenance, retraction) is this project's whole identity.

The bootstrap populate run already recorded this ceiling empirically: all 12 approved connections were Torah Ohr ↔ Torah Ohr, none reaching Tanya, because similarity-based mining pairs within the larger book's neighborhood (POPULATE_REPORT.md, "cross-*parsha*, not yet cross-*work*"). The fix is not a better similarity — it is a different kind of object.

Anchor: `linked_refs: list[str]` in `maayan/capture/models.py:48` — the current system's *entire* representation of "these texts are related": an untyped list of strings inside a prose chunk.

Hands-on: take one connection you (or the bootstrap) have made and write it three ways on paper: (a) as the prose chunk it is today; (b) as `(subject, object)` — a bare pair; (c) as `(subject, rel_type, object, evidence, author)`. Now answer, for each form: can a program find all connections *of this kind*? Can it detect a conflicting one? Can it say *how it knows*? Only (c) answers all three. You have just derived Prompt 33.

Lesson 0.3 — The credo and the map

> Module 0, Lesson 3 · ~12 min. The one question this answers: **what single design rule generates all ten prompts of Phase 7 — and how do the pieces depend on each other?**

The Phase 7 plan compresses its philosophy into one line, and this course will return to it in nearly every module, so learn it now:

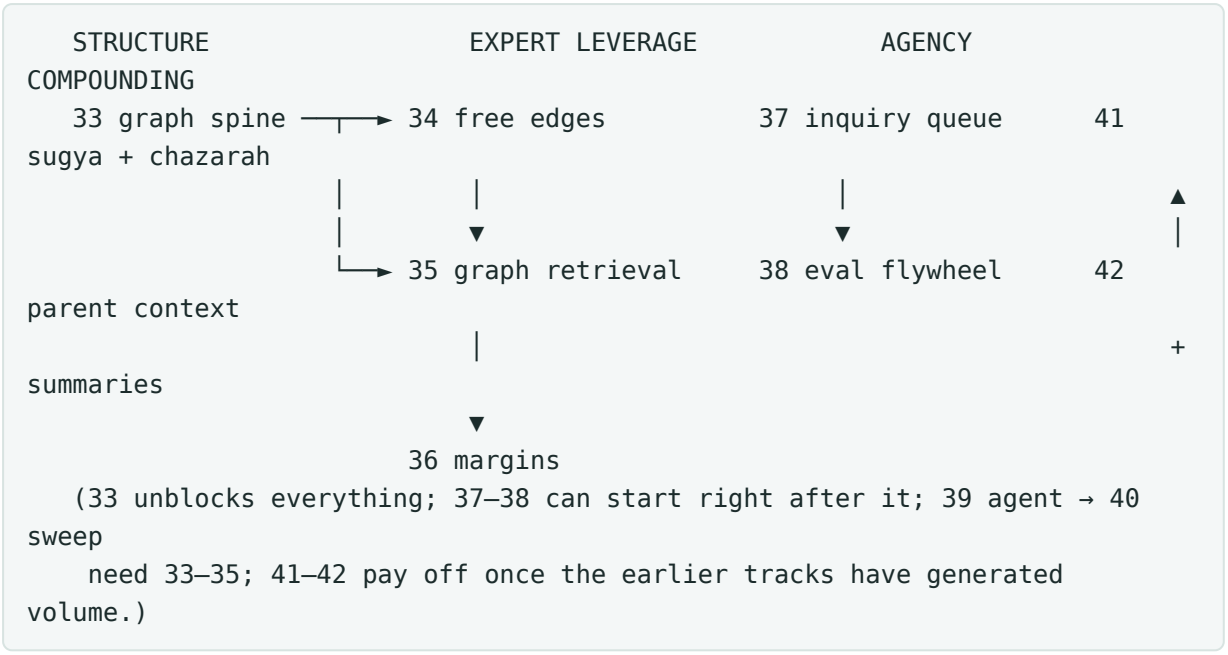
> **Typed knowledge + strong verifiers + bounded agency + a human gate = a harness in which a better model converts directly into more approved knowledge.**

Unpack each term:

- **Typed knowledge** — assertions stored in a form a program can check, walk, and compose (Modules 1–2), not prose a program can only re-embed.
- **Strong verifiers** — machine checks that catch bad work regardless of who produced it: the faithfulness pass, citation verification, graph consistency, the test suite. Verifiers are what let you delegate. This is the deepest lesson the project borrows from elsewhere: Mathlib grows because Lean *checks* every proof, so contributions can be accepted from anyone — the checker, not the prover, is the asset (Module 5).
- **Bounded agency** — the model may act, but inside a small, typed action space with hard budgets (Module 5), so a stronger model does *more* within the same guarantees.
- **A human gate** — the Rav's approval remains the only path into the index for anything model-drafted. Phase 7 does not weaken the gate; it *aims* it (Module 4).

The right-hand side of the equation — "a better model converts directly into more approved knowledge" — is called **elasticity**, and it is the phase's chief metric (Module 8). A harness with low elasticity wastes the future: model quality rises around it and its output stays flat. A harness with high elasticity is exactly what the mission statement asks for: *a harness for future good AI models*.

The ten prompts, as a dependency map:



Anchor: BUILD_PLAN_PHASE7.md §§0–1 (the assessment, the lessons-from-other-fields table, the credo) and §5 (sequencing).

Hands-on: read the plan's §1 table of borrowed patterns — legal citators, UniProt, Mathlib, the daf, Anki, Community Notes. For each row, say aloud in one sentence what maayan takes from it. If you can do that unaided, you hold the whole phase in your head; everything after this is detail.

Module 1 — Typed knowledge: the graph spine

Goal: understand graphs from zero, why the TYPE of an edge carries most of its value, how evidence codes record "how we know," and how the spine stays honest through gates — the content of Prompt 33.

Lesson 1.1 — A graph, from first principles

> Module 1, Lesson 1 · ~10 min. The one question this answers: **what is a knowledge graph, really — and what queries does it make possible that a vector index cannot answer at any price?**

Forget the buzzword. A graph is two lists:

- a list of **nodes** — things that can be talked about;
- a list of **edges** — assertions of the form *(this node, relates-thus, that node)*.

That is all. Its power is not in the storage — SQLite tables hold it happily, no graph database required — but in the **queries the shape makes cheap**:

- **Neighborhood:** "everything asserted about Tanya ch. 18" is one indexed lookup, not a similarity search that might or might not surface it.
- **Walking:** "what does the thing connected to this thing connect to?" — follow edges hop by hop. This is what a scholar does with mareh mekomos, and what Module 3 teaches retrieval to do.
- **Joining:** two edges sharing an endpoint imply a candidate relationship between their far ends. Deterministic, auditable composition (Lesson 2.2).
- **Consistency:** two edges of *conflicting types* between the same endpoints is a contradiction you can *detect with a query* — and hand to the Rav as a question (Lesson 4.2).

In maayan's graph (Prompt 33) the nodes are of two kinds: **refs** (canonical citations — `"Tanya, Part I; Likkutei Amarim 18"` — the same strings the whole system already lives on) and **concepts** (canonical ideas — ביטול, צמצום — which Lesson 1.4 treats). The edges are `Relation` rows. Note the deliberate humility of the storage decision: the same SQLite file that holds chunks holds relations. Phase 7 adds no infrastructure — it adds *shape*.

Anchor (after Prompt 33): `maayan/graph/models.py` (`Concept`, `Relation`, `Endpoint`) and `maayan/graph/` store queries `relations_for` / `neighbors`.

Hands-on: on paper, draw five nodes: Tanya 18, Torah Ohr Vayera, the pasuk "וירא אליו ה׳", the concept אהבה בתענוגים, and one shiur. Draw the edges you believe are true, and label each with a verb. You have just experienced the central discipline of this module: the moment you must *choose a verb*, you are forced to know what you actually mean. Prose lets you not choose. The graph does not.

Lesson 1.2 — The type is the value

> Module 1, Lesson 2 · ~15 min. The one question this answers: **why is a typed edge worth an order of magnitude more than a bare link — and what exactly does each of our ten relation types assert?**

The strongest precedent for Phase 7 comes from law. A legal citation index that merely says *case A cites case B* is a curiosity. Shepard's Citations — the tool without which American lawyers literally would not practice — says *how*: A **followed** B, A **distinguished** B, A **overruled** B. The type is the value: it tells you whether B is still good law. One bit of linkage, plus a verb, becomes professional infrastructure.

The Torah world built the same thing into the printed page centuries earlier: the Mesoras HaShas tells you where this Gemara's language recurs; Ein Mishpat tells you where it is *codified* — a typed edge to halacha; Toras Ohr (the margin, not the sefer) tells you the pesukim. Different verbs, different margins. Phase 7's apparatus (Lesson 3.3) is this page, machine-assembled.

Prompt 33 fixes ten verbs. Learn them as assertions, each with its Torah-shaped meaning:

`rel_type`	The assertion, in a sentence
`quotes`	Subject explicitly cites object (a Zohar, a midrash, a ma'amar).
`quotes_pasuk`	Subject explicitly cites this verse of Tanach. The workhorse — see Lesson 2.2.
`explains`	Subject gives the perush of object — object is primary, subject serves it.
`is_mashal_for`	Subject is the analogy whose nimshal is object. Deeply chassidus-specific: meshalim travel between ma'amarim and finding one is finding gold.
`corresponds_to`	Systematic mapping: a Name to its sefirah (חכמה ↔ ע"ב), a level to its olam. The Kabbalistic grid itself.
`argues_with`	Subject and object are in real dispute — a machlokes, preserved as a machlokes.
`develops`	Subject takes object's idea further (the Mitteler Rebbe developing the Alter Rebbe).
`parallels`	Same inyan treated independently in both — neither quotes the other.
`defines`	Subject is where object (a concept) receives its definition.
`connects`	The honest residual: related, per the asserter, in a way none of the above captures.

Two design notes worth internalizing. First, `connects` is deliberately the *weakest* type — it is where the existing prose connections land when Prompt 33's backfill lifts them, because they never had to declare a verb. Expect the inquiry queue (Module 4) to eventually ask the Rav to *upgrade* important `connects` edges to real types. Second, the vocabulary itself is **schema** — the mold decades of knowledge will be poured into — which is why the plan's execution protocol (Module 9) makes it the one part of Prompt 33 a top model reviews with the Rav before merge.

Anchor: the `rel_type` Literal in Prompt 33's spec (BUILD_PLAN_PHASE7.md, Prompt 33).

Hands-on: take the twelve bootstrap connections from POPULATE_REPORT.md and, for any three, decide which of the ten types each *should* be if a scholar reviewed it. Notice how often you want to say "probably `parallels`, but I'd have to check both sources" — that sentence is precisely an inquiry-queue item (Module 4), and you just generated it by hand.

Lesson 1.3 — How we know what we know: evidence codes

> Module 1, Lesson 3 · ~12 min. The one question this answers: **why does every assertion carry a code for how it came to be believed — and what does each code license the system to do?**

Biology solved a problem we have. The protein databases hold millions of annotations — "this gene does X" — produced by wildly different means: a wet-lab experiment, an inference from a similar sequence, a computational prediction no human ever checked. Treating those as equally true would poison everything downstream, and *deleting* the uncertain ones would discard most of the map. The solution (UniProt, the Gene Ontology) is the **evidence code**: every assertion permanently tagged with *how it is known*, so downstream consumers choose their own floor of rigor.

Phase 7 adopts this wholesale. Every relation (and, extended by Prompt 36's spirit, every layered chunk) carries one of five codes:

`evidence`	How it came to be	Example
`expert`	A named scholar asserted it	The Rav's Tanya↔Likutei Torah connection
`curated`	Human-curated data ingested from a trusted source	A Sefaria link (Lesson 2.1)
`deterministic`	Computed by a rule with no judgment in the loop	A pasuk-join edge (Lesson 2.2)
`derived_approved`	Model-drafted, faithfulness-checked, human-approved	A top-model sweep edge the Rav accepted
`derived_bootstrap`	Same pipeline, free-model era, historically distinguishable	The 28 bootstrap items

Read the table again and notice it is *not* a ranking from good to bad in any simple sense — `deterministic` edges are in one way the most trustworthy of all (a rule either fired or it didn't), yet they assert the *least* (a shared pasuk, not an insight). The code answers "how do we know," and different questions demand different floors: retrieval boosting might favor `expert`; the benchmark's ablation *requires* that `derived_bootstrap` be separable (this is why the bootstrap run recorded its author as `bootstrap:gpt-oss-120b:free` — the code existed avant la lettre); and the chazarah scheduler (Module 6) targets low-evidence items for re-review *because* the code marks them.

The deep point: **evidence codes are what let a knowledge base be honest about itself.** Without them, one wrong bootstrap edge discredits the whole graph; with them, the graph can say "here is what a scholar staked his name on, here is what a rule computed, and here is what a free model once guessed and a human waved through — treat each accordingly."

Anchor: the `evidence` Literal in Prompt 33; `confidence: float` beside it (a *within-code* refinement — the two are not redundant).

Hands-on: classify these five by code: (1) "Tanya 18 quotes Tehillim 73:26" found by the Sefaria links ingest; (2) the same fact typed in by the Rav; (3) "Torah Ohr Miketz and Mishpatim both quote Tehillim 73:26 → parallels" computed by the join; (4) a sweep model's proposal that Miketz *develops* Mishpatim, approved by the Rav; (5) the same proposal, still awaiting review. (Answers: curated, expert, deterministic, derived_approved — and (5) has *no* standing at all: `status="proposed"` means the retrieval layer must behave as if it does not exist.)

Lesson 1.4 — Two kinds of nodes, and the gate as a type system

> Module 1, Lesson 4 · ~12 min. The one question this answers: **why do concepts get their own node type instead of being strings — and how do the gates make the whole spine trustworthy?**

Refs name *places in text*. But half of what a scholar asserts is about *ideas*: ביטול is defined here; צמצום corresponds to that; this chapter explains אהבה בתענוגים. Ideas have a property refs don't: **many names for one thing** — ביטול במציאות ,ביטול, bitul, ביטול היש — and if each spelling is its own string, the graph fragments and neighborhood queries silently miss most of what they should find. So Prompt 33 gives concepts first-class nodes: a canonical name, aliases, a `kind` (sefirah / partzuf / olam / name / avodah / concept), and — crucially — an optional link to the **lexicon term** the system already maintains. The lexicon, which Course 2 taught as a query-expansion trick, is quietly becoming an *ontology*: the curated registry of what exists in this domain and what it is called. That promotion, not any single feature, is Prompt 33's deepest consequence.

Technically, an edge endpoint must therefore be "a ref *or* a concept" — and the plan insists this be a **discriminated union** (an explicitly tagged either/or type), not a bare string that might be either. First principles: when a value can be two things, make the type say *which*, and every function that touches it is forced to handle both — the compiler becomes a verifier. That is the house rule "typed throughout" doing real work, and you will see the same move in the agent's action type (Lesson 5.2).

Finally, the gates. A relation is born `proposed`. Only an explicit approval — an author, a timestamp — makes it `approved`; only approved relations are ever served to retrieval, the apparatus, or the agent. Retraction goes through the *existing* audited retract flow (who / when / why), never a bare DELETE. Notice what this buys: the graph can accept *proposals from anyone* — a free model, a sweep, a well-meaning student — at zero risk, because standing is granted only at the gate. In the type-system analogy that runs through this whole project: **`proposed` → `approved` is a cast that only a human can perform.** Everything in Modules 4 and 5 leans on that being literally, mechanically true.

Anchor: Prompt 33's `Endpoint` union, `Concept.kind`, `status` lifecycle; the backfill (`maayan graph backfill`) that lifts existing prose connections into `connects` edges *without deleting the prose* — the chunk keeps carrying the insight's words; the edge carries its shape.

Hands-on (after Prompt 33): run `maayan graph backfill`, then `maayan graph show --ref "Tanya, Part I; Likkutei Amarim 18"`. Every line you see was already in the system — as prose. What is new is only that a program can now *answer questions about it*. Then try to approve a relation without an author and watch it refuse. That refusal is the whole module in one line of output.

Module 2 — Edges for free: curated links and the pasuk as primary key

Goal: learn the acquisition-cost hierarchy of knowledge, why thousands of high-trust edges are available for zero model spend, and how a verse functions as a database join key across the entire Torah corpus — the content of Prompt 34.

Lesson 2.1 — Deterministic beats generated

> Module 2, Lesson 1 · ~10 min. The one question this answers: **before asking any model to propose knowledge, what should we always do first — and why is the cheap path also the more trustworthy one?**

There is a hierarchy in how an assertion can enter a knowledge base, and it runs opposite to the industry's instincts:

1. **Ingest what humans already curated.** Sefaria maintains a link graph — years of human editorial work connecting texts to the pesukim and sources they cite. For our corpus's books, those links exist *as data*, one API call away. Re-deriving them with a language model would be strictly worse on every axis: cost, error rate, auditability. (Evidence code: `curated`.)
2. **Compute what rules can compute.** Some relationships follow mechanically from data we already hold — Lesson 2.2's pasuk join. No judgment, no model, no gate anxiety. (`deterministic`.)
3. **Only then, ask a model** — grounded, faithfulness-checked, gated. (`derived_*`.)
4. **And treasure above all what only the scholar knows** — the connections in no database and derivable by no rule. (`expert`.)

The strategic insight is that steps 1–2 are not a poor man's substitute for 3–4. They are the **scaffolding that makes 3–4 work better**: a model proposing connections over a graph that already knows every pasuk-anchor has vastly more to stand on than one staring at raw text, and the Rav reviewing its proposals can see the deterministic context around each. The bootstrap report's central disappointment — no cross-work connections — falls *immediately* to step 2, before any funded model run: Tanya and Torah Ohr quote the same pesukim constantly, and shared pesukim are computable.

OpenStreetMap is the precedent worth knowing: its richness comes from mass imports of existing government data *plus* volunteer judgment *plus* automated validators — three sources, one map, provenance kept. One map, provenance kept, is exactly Phase 7.

Anchor: Prompt 34 (`maayan graph ingest-links`, the pasuk join); `config.sefaria_link_categories` — the kept link kinds are configuration, not code.

Hands-on: fetch one ref's links yourself and look at real payload: `curl -s "https://www.sefaria.org/api/links/Tanya%2C%20Part%20I%3B%20Likkutei%20Amarim.18" | head -c 2000` (shape may differ — the prompt requires the builder to verify against live data before writing the parser, and to mock it in tests; both halves of that sentence are house rules you already know).

Lesson 2.2 — The pasuk is the primary key

> Module 2, Lesson 2 · ~12 min. The one question this answers: **what single fact about how chassidus is written makes the whole corpus a graph waiting to be read off — no model in the loop?**

In a relational database, two tables that seem unrelated are joined the moment they share a key. The deepest structural fact about Torah literature is that it *has* a natural key, and every stratum of the tradition uses it: **the pasuk.** A ma'amar in Torah Ohr does not begin from nowhere — it opens on a verse (וירא אליו... ראו כי ה׳ נתן לכם השבת), and its argument returns to verses at every turn. Tanya's chapters are laced with them. The Zohar is *organized* by them. When the Alter Rebbe and the Gemara and the Midrash all speak about ראו כי ה׳ נתן לכם השבת, they are — by construction, before any judgment is applied — speaking about a common thing.

So the join: after Prompt 34's ingest gives us `quotes_pasuk` edges (our-ref → pasuk), any two corpus refs sharing a pasuk target receive a `parallels` relation, evidence `deterministic`, author `system:pasuk-join`, with the shared verse recorded in the note. Read the properties off:

- **It is cross-work by nature.** The verse doesn't care which sefer quotes it. The very reach the similarity-based bootstrap could not achieve — Tanya ↔ Torah Ohr — falls out of a join over curated data, at $0.
- **It is honest about what it asserts.** `parallels`-via-pasuk claims only "both stand on this verse" — a *candidate* affinity, exactly the right strength. Whether the two drushim actually illuminate each other is scholar's work; the edge's job is to make sure the question gets *asked* (the inquiry queue can promote strong pasuk-parallels for review).
- **It is idempotent and dedupe-safe** — rerun changes nothing, because the store dedupes on (subject, object, type). Pacing rides the injected `Clock`, resumability rides a cursor table, both patterns lifted verbatim from the populate drip. New machinery: none.

A discipline note: it is tempting to extract pasuk citations from the Hebrew text directly with regexes. The plan deliberately does *not* start there — quoted fragments, partial verses, and rashei-teivos make

text-mining a judgment problem in disguise. Start from curated link data; let text-mining be a later, gated enhancement if the coverage gap justifies it. Knowing what *not* to build is a Phase 7 skill.

Anchor: Prompt 34's mapping rules (Tanach target → `quotes_pasuk`; join → `parallels`), and its test fixtures — one real captured payload, then network mocked forever after.

Hands-on (after Prompt 34): run the ingest with `--limit 5` on Tanya, then `maayan graph show --ref` on one of those chapters. Count the edges. Multiply by the corpus size. That number — thousands — cost nothing, and every one of them is a place the apparatus (Lesson 3.3), the retriever (Lesson 3.1), and the agent (Module 5) can now stand.

Module 3 — Retrieval that walks, and the margins of the daf

Goal: understand how retrieval uses the graph without moving the trust core one inch, and how the same graph gives every source its margins — Prompts 35 and 36.

Lesson 3.1 — Recall finds doors; the graph opens rooms

> Module 3, Lesson 1 · ~12 min. The one question this answers: **what exactly happens, step by step, when graph expansion is on — and why is one hop enough?**

Vector search and graph traversal are good at opposite things, and the design pairs them so each does only what it is good at. Hybrid search has **recall**: from a free-text question, it finds *entry points* — passages whose wording or meaning matches. It knows nothing of structure. The graph has **structure**: from any entry point, it knows what a scholar asserted stands nearby. It cannot process a question at all. So:

1. Run the inner retriever exactly as today (including Course 2's expansion, if on).
2. **Decide the default-deny gate on those seed scores.** (Lesson 3.2 — the load-bearing line.)
3. If and only if the gate passes: for the top few seeds, fetch **approved** relations touching their refs; resolve the far endpoints to chunks; also expand from concepts the lexicon detects in the query (a question about ביטול pulls the `defines` edge's target even if wording missed it).
4. Under a hard budget (`GRAPH_EXPAND_LIMIT`), merge expanded chunks into the candidate set, marked with their **via** — the relation that brought them — and rerank together.

Why one hop and not a grand traversal? Because each hop multiplies candidates and dilutes relevance; because the reranker can adjudicate a merged set of dozens, not thousands; and because multi-hop *reasoning* is the agent's job (Module 5), where each hop is *chosen* rather than exhaustively taken. One deliberate hop from strong seeds captures the overwhelming share of the value — the expert connection, the pasuk-sibling, the definition — at bounded cost. Resist, permanently, the urge to raise the hop count before measurement (Module 8) says the first hop paid.

The **via** field deserves respect: when an answer cites a source that arrived by expansion, the scholar sees *"↳ via connects (R. Ginsburgh) from Tanya 18"*. Provenance of retrieval, not just of text. In a system whose product is trust, how a source got to the table is part of the answer.

Anchor (after Prompt 35): `GraphExpandingRetriever` behind the existing `Retrieving` protocol — study how it wraps the inner retriever exactly the way Course 2's

`MultiQueryRetriever` did; the factory composes them. Nothing downstream changes: that is what "protocol" has meant since Course 1.

Hands-on (after Prompt 35): pick a question whose answer should span Tanya and Torah Ohr. Run it with `GRAPH_EXPAND_ENABLED=false`, then `true`, and diff the source lists. Find the via line. Then run `make eval` — Prompt 35 extends the crosstext harness so this improvement is a *number*, not an anecdote.

Lesson 3.2 — The gate does not move

> Module 3, Lesson 2 · ~10 min. The one question this answers: **why must graph expansion never rescue a refusal — and what breaks, precisely, if it does?**

Here is the subtlest invariant in Phase 7, and the one a hurried implementation is most likely to violate. State it first: **the refusal decision is made on the seed retrieval scores alone, before the graph is touched. Expansion may enrich a supported answer; it may never convert a refusal into an answer.**

Derive it, don't memorize it. What does the default-deny gate actually assert? That *the corpus contains material relevantly similar to this question* — measured by retrieval score against a threshold. That assertion is the entire foundation of "maayan never answers from a model's memory." Now suppose expansion ran before the gate: a question with *no* relevant seeds — a question the corpus cannot answer — could still accumulate chunks, because even weak seeds have neighbors, and neighbors have neighbors. The gate would then be comparing the threshold against material that arrived by *association with irrelevance*. The refusal guarantee wouldn't be weakened; it would be **meaningless** — every question "passes" eventually, and the system is back to confident answers built on nothing, wearing citations as decoration. The single most valuable property of the whole project, gone, with no test failing loudly.

Hence the implementation discipline in Prompt 35: gate first, on seed scores, using the *existing* gate code (never a reimplementation — a second gate implementation is a second place for this bug to live); expansion only on the true branch; and a test that *injects a spy* to prove a refused query never touches the graph store. When you review that PR (Module 9), you will check this with your own eyes regardless of what the tests claim. It is also why the plan routes Prompts 35 and 39 — the two that stand nearest this cliff — to the deepest review tier.

Anchor: the gate in `maayan/generate/rag.py` (Course 2, Lesson 2.4 — "the rule that didn't change"); Prompt 35's requirement 2 and its spy test.

Hands-on: write, in two sentences of your own words, what a user would experience if this invariant broke — and why no ordinary accuracy metric would catch it quickly. (The answers the system

should refuse are precisely the ones with no gold sources to grade against.) Keep your two sentences; they belong in your review checklist.

Lesson 3.3 — The margins: an apparatus for every source

> Module 3, Lesson 3 · ~10 min. The one question this answers: **what should a scholar see when standing on a passage — and why is a read-only view the most beis-medrash-like feature in the whole phase?**

Open a daf of Gemara. The text occupies the middle; around it, the accumulated centuries: Rashi explaining, Tosafos arguing, Mesoras HaShas pointing to parallel sugyos, Ein Mishpat pointing into halacha. The page *is* a user interface, arguably the most successful knowledge UI ever built, and its principle is: **stand anywhere, and see everything the tradition knows about where you stand — attributed, typed, and layered.**

Prompt 36 builds that page for maayan. For any ref, the apparatus assembles, read-only:

- **Relations**, grouped by type — who quotes it, what pesukim it stands on, what explains it, what argues, each edge with author and evidence badge;
- **Terms** — lexicon entries whose surface forms occur in the passage (reusing the lexicon's existing matcher — no new matching engine, per the standing rule);
- **The expert layer** — connections, developments, shiur segments grounded on this ref;
- each item a click-through to its own full context.

Notice three things. First, it writes nothing — it is a *view* over stores that already exist, which is why it is cheap and safe and why the plan sequences it right after the free-edges prompt fills the graph with something to show. Second, it changes the scholar's experience more than any retrieval feature: search serves the question you had; margins serve the question you *didn't know to ask* — you look up a chapter and discover a shiur touched it. Third, it is the capture loop's shop window: when the Rav sees his own connection sitting in the margin of Tanya 18 like a Tosafos, typed and attributed, the value of every future contribution becomes self-evident. Adoption is a feature.

Anchor (after Prompt 36): `ApparatusService`; `GET /api/apparatus?ref=…`; `maayan apparatus <ref>`.

Hands-on (after Prompt 36): open Tanya ch. 18 in the reader, tap "margins ▸", and read the page like a daf. Then ask: what is missing that a real daf would have? Your answer is a candidate inquiry item or a Phase 8 note — write it down.

Module 4 — The talmid asks: attention, questions, and the eval flywheel

Goal: treat the Rav's hour as the system's scarcest resource, learn where machine- generated questions come from and how they are ranked, and close the loop that turns every approval into a permanent test — Prompts 37 and 38.

Lesson 4.1 — The binding constraint

> Module 4, Lesson 1 · ~10 min. The one question this answers: **what actually limits how fast this system's knowledge can grow — and what follows from naming it honestly?**

Every system has one binding constraint, and optimizing anything else is decoration. Walk the candidates: Compute? The funding brief prices a *full* top-model populate run at tens of dollars — a rounding error. Text? The corpus grows by adapters, a solved pattern. Model quality? Rising on its own, yearly, for free. What remains is the input that cannot be bought, scaled, or waited into existence: **the scholar's attention.** Every path into the knowledge base — annotation, term, development, relation, sugya page — passes through the gate, and the gate is a human hour.

Name that constraint and the design consequences fall out one by one:

- **Never spend the hour on triage a machine can do.** Faithfulness checks run before anything reaches the queue; ungrounded drafts die unseen (the bootstrap's 30 default-denials cost the Rav nothing).
- **Rank what does reach him by expected value.** A flat chronological queue treats a question three learners asked and got refused identically with a duplicate low-value edge. Prompt 37's priority score is crude and transparent by design — but it exists, and *that* is the innovation.
- **Bank every answer twice.** An hour that produces an approval should also produce a regression test (Lesson 4.3) — the same hour, spent once, paying forever.
- **Measure leverage.** Approved knowledge items per expert-hour is a north-star metric (Module 8) precisely because it prices the constraint.

This reframing has a name in the literature — *active learning*: when labels are expensive, the system should choose which questions to ask, not passively receive whatever comes. But the beis medrash knew it first: a good talmid is measured by his questions, and the rebbe's time flows to the talmid who asks the ones that matter. Phase 7 makes maayan that talmid.

Anchor: BUILD_PLAN_PHASE7.md Prompt 37; the north-star metrics table in its §1.

Hands-on: estimate, honestly, the Rav's available hours per week for review. Multiply by four weeks. That integer is Phase 7's real budget — not the dollar figures. Keep it in view whenever you are tempted to build a generator that emits hundreds of low-value items.

Lesson 4.2 — Where good questions come from

> Module 4, Lesson 2 · ~14 min. The one question this answers: **what are the four places the system finds questions worth the Rav's time — and why is each one trustworthy as a signal?**

A question-generating system lives or dies by its sources of signal. Prompt 37 draws from four, each with a different epistemic character:

1. **Refused questions** (`refused_question`). Until Phase 7, a refusal was correct behavior and *discarded evidence*. But look at what a refusal actually is: a user revealed what they need, and the system confessed it cannot serve it. Logged (`AskLog`), clustered (near-duplicates merged), and counted, refusals become the most honest gap-map possible — demand-weighted, not speculation. "Three learners asked about ביטול this week; the corpus refused every time" is a question that *arrives pre-justified*.
2. **Under-grounded terms** (`undergrounded_term`). The bootstrap's deny-list, group 2: core terms (אתכפיא, בינה, כתר, the Names מ"ה/ס"ג/ע"ב) that a free drafter failed to ground but which the corpus plausibly supports. Each is a crisp, closed question — "confirm or dismiss" — ideal for the minutes between bigger reviews.
3. **Low-evidence knowledge** (`low_evidence_relation`). Everything Module 1's evidence codes marked as weakly known — bootstrap-era edges, low-confidence relations — is a standing invitation for an upgrade. The code *is* the queue filter.
4. **Contradictions** (`contradiction`). The graph makes conflict computable: two approved edges sharing endpoints where one says `argues_with` and another says `explains` is a machine-detectable tension. And here, borrow Community Notes' deep insight: **disagreement is precisely where human judgment adds the most value** — so route it there. Version 1 surfaces candidates without judging them; "Rav, is this a real machlokes or two madreigos?" is *exactly* a beis medrash question, and only he can answer it.

Two rules govern the whole queue. **Transparency:** the priority score is a documented, config-weighted blend whose components print with the item — the Rav can always see *why* something ranked. Black-box ranking of a scholar's attention would be obnoxious. **Closure:** answering deep-links into the existing capture flows (annotate / add-term / approve / retract) — the queue creates no new write path, hence no new gate to audit — and the resolution records what was created. Dismissal is a first-class outcome: "not worth my time" is information, and generators that collect dismissals should learn to be quieter.

Anchor (after Prompt 37): the four generators in `maayan/inquiry/`; the typed per-kind payloads (a discriminated union again — Lesson 1.4's move, third appearance).

Hands-on: before the code exists, be the generator. From the populate report's deny-list and your own refused questions this month, hand-write the top five inquiry items with a one-line justification each. Keep the list; when Prompt 37 lands, compare its output to yours. Where the machine's list is worse, you have found either a weight to tune or a generator to add — file it.

Lesson 4.3 — Every approval becomes a test

> Module 4, Lesson 3 · ~12 min. The one question this answers: **what does software engineering's oldest discipline — every bug fix ships with a test — look like when the "code" is knowledge?**

Why do mature codebases demand a test with every bug fix? Because a bug that happened once is *proven possible*, and a fix without a test is a fix that can silently un-happen. The test suite is the codebase's memory of everything that ever went wrong — which is why it only ever grows, and why regressions get caught by the fossil of the original incident.

Now translate. When the Rav approves a connection, he has certified: *this question has this answer, grounded in these refs.* That is — read it again — a **gold eval example**: question, expected refs, gold answer. Today that certification improves retrieval once and evaporates as evidence. Prompt 38 keeps it: on every approval (connection, development, term, answered inquiry), a `CapturedExample` is drafted **deterministically — by template, zero model calls**: a connection between X and Y yields "מה הקשר בין X ל-Y?" with its linked refs as `expected_refs` and the approved body as the gold answer; a development's seed directive (already phrased as an instruction) becomes the question; a term yields "מהו X?". Mechanical, auditable, free.

`maayan eval --captured` then runs the standard retrieval harness over the accumulated set, emits a date-stamped JSON report, and prints the delta against the previous run. The consequences compound quietly:

- **Regressions become visible.** Re-chunk the corpus, swap an embedder, tune a threshold — and if the Rav's connection from last month stops being retrievable, a number drops *that day*, naming the exact example.
- **The gold set stops being a bottleneck.** The benchmark plan hinges on ~60 hand-written expert questions — they remain the *exam*. But the captured set is the *homework*, growing by itself, hundreds strong within months, at zero marginal cost.
- **Hygiene is symmetrical.** Retract the origin and the example retracts with it — the flywheel must never fossilize withdrawn knowledge as a test that enforces it.

One number worth internalizing from Module 8 in advance: the **compounding rate** — the fraction of new approvals grounded partly on *prior* layered knowledge. The flywheel is its instrument: when captured examples start citing sugya pages citing expert edges, the loop is feeding itself, and that curve is the project's real product.

Anchor (after Prompt 38): the injected recorder on the approval paths (one hook point, not logic scattered through services — review for exactly that); `eval/captured_goldset` store; the retract subscription.

Hands-on (after Prompt 38): approve one connection, run `maayan eval --captured`, and find your example in the JSON. Then retract the connection and run again. Watching an eval case appear and disappear with its knowledge teaches the architecture faster than any diagram in this course.

Module 5 — The chavrusa agent: bounded agency

Goal: strip the word "agent" to its mechanical core, understand why the action space — not the model — is the capability, see how verifiers make delegation safe, and scale it into corpus-wide study programs — Prompts 39 and 40.

Lesson 5.1 — What an agent is, stripped of hype

> Module 5, Lesson 1 · ~10 min. The one question this answers: **mechanically — no metaphors — what is an agent loop, and what specific inability of the current system does it remove?**

Course 2 ended by naming the horizon: "agentic multi-hop... needs a richer contract" (Lesson 5.3 there). Here is the whole machinery, demystified. An agent is a loop with four parts:

- **State:** what has been gathered so far — here, the *working set* of retrieved chunks, plus the trail of actions taken.
- **Actions:** a fixed menu of things that can be done next — here, exactly five (Lesson 5.2).
- **A policy:** something that looks at state and picks the next action — here, the model, via one constrained-JSON completion per hop.
- **Budgets:** hard limits — hops, chunks, model calls — whose exhaustion forces the loop to conclude with whatever it has.

Nothing else. No autonomy mystique: a while-loop, a menu, a counter.

What it removes: every smart verb in the system today is **one-shot** — one retrieval, then generation, with no way to act on what the sources revealed. But watch a real chavrusa study: read a passage → *it names a concept* → look the concept up → *the definition points to another ma'amar* → open it → search again with better words → now answer. Each step's input is the previous step's output. That see-something-follow-it structure is not exotic; it is *iteration*, and the current pipeline simply has no place for it. The chavrusa agent gives it one, bounded.

And the invariant, before anything else: **the gate still fires first.** The seed retrieval and default-deny check happen before the model is consulted at all — a refused question never wakes the agent. The loop makes supported answers *deeper*; it gives unsupported questions nothing. Same sentence as Lesson 3.2, because it is the same law.

Anchor (after Prompt 39): `maayan/agent/service.py` — find the four parts in the code: the working set, the action union, the completion-parse step, the budget checks. They will be exactly

where this lesson says.

Hands-on: run `uv run maayan ask --reason` on a hard cross-text question and read the study map. Write down the *one action* you wish the system could have taken after reading its own map ("go look up that term it mentioned", "open the chapter before"). That wish, typed, is a `StudyAction` — and Lesson 5.2 is the discipline of choosing which five wishes to grant.

Lesson 5.2 — The action space is the capability

> Module 5, Lesson 2 · ~14 min. The one question this answers: **why do five verbs and strict JSON beat a rich tool API — and why is the action space the part worth designing hardest?**

A finding from the agent-research literature, learned expensively by everyone who builds these systems: **agent performance is a function of the interface you give the model at least as much as of the model itself.** Give a strong model a sprawling, ambiguous tool surface and it flails; give a modest model a small, crisp, well-described action space with informative results, and it performs beyond its weight. The interface is the capability. (The research name is *agent-computer interface*, ACI; the plan's §7 ask 5 applies it recursively — a top model critiquing the harness it worked in.)

maayan's chavrusa gets exactly five verbs:

Action	The chavrusa move it encodes
`Search(query)`	"Let me look for where this is discussed" — re-retrieve with better words. Results below the score threshold add nothing: the gate's semantics apply to every increment, not just the seed.
`OpenSection(ref)`	"Show me the whole perek" — context around a hit, by `section_path`.
`WalkGraph(ref)`	"What does the tradition connect to this?" — approved edges only, the Rav's teaching becoming a *road* the model can take.
`LookupTerm(term)`	"What does this word mean here?" — the lexicon as reference desk.
`Conclude()`	"I'm ready to say over what I found."

Why not more? Each verb added multiplies the ways a hop can go sideways and dilutes the model's grasp of every other verb. These five cover the chavrusa's actual repertoire; anything else — proposing edges, drafting terms — is a *write*, and writes belong to gated flows, not to a loop answering a question. The action union is a discriminated type (Lesson 1.4's move, again): an action either parses into exactly one legal verb with legal arguments, or it is rejected.

Why constrained JSON over a provider's tool-calling API? Because the `GenerationBackend` protocol is the house's load-bearing wall: OpenRouter, Ollama, a local model, next year's backend — all speak "text in, text out." Building the agent on plain completions plus strict parsing keeps *every* backend eligible, which is the elasticity credo in engineering form: never marry the harness to one vendor's calling convention. The cost is that models sometimes emit malformed actions; the protocol prices that in — **one repair retry, then force-conclude with what has been gathered.** Degradation, never a crash, never a bypass.

And the trace. Every step — action, arguments, result digest, budget spent — persists as a typed `StudyTrace`. Read it as a teacher reads a talmid's notebook: *how* did it get to this answer? Which edge did it walk (and who taught that edge)? Where did it waste a hop? Traces are the debugging surface, the demo's centerpiece, and — run the same question across two models — the raw material of the elasticity measurement.

Anchor (after Prompt 39): `StudyAction` / `StudyStep` / `StudyTrace` in `maayan/agent/models.py`; the repair-retry branch; `ask --agent --show-trace`.

Hands-on (after Prompt 39): run `ask --agent --show-trace` on the question from Lesson 5.1's hands-on. Check: did the loop take the action you had wished for? Then find in the trace the moment a budget or a below-threshold search *stopped* something — the guardrails are most educational where you can see them bite.

Lesson 5.3 — Verifiers let you trust stronger models

> Module 5, Lesson 3 · ~10 min. The one question this answers: **what, precisely, makes it safe to hand this loop a much stronger model and simply let it study harder?**

Here is the question a skeptic should ask about Module 5: "you've built a loop where a language model makes choices — why isn't that exactly the uncontrolled-model risk this project exists to prevent?" The answer generalizes far beyond this repo, so get it exactly right.

Consider Mathlib, the great library of formalized mathematics. It accepts proof contributions from students, hobbyists, and — increasingly — language models, and its standards never waver. Not because contributors are vetted, but because **Lean checks every proof mechanically.** The prover can be anyone or anything; the *checker* is the asset. Trust lives in verification, not in the goodness of the generator.

Now audit the chavrusa loop against that standard, and notice it is verified at every boundary:

- **Before:** the default-deny gate — no support, no loop (Lesson 5.1).
- **During:** every action parses against a closed union or is rejected; every search increment passes the same score threshold; every budget is enforced in code, and exhaustion force-

concludes.
- **After:** the final answer runs the *existing* synthesis path and the *existing* citation-verify pass — the same checks a non-agent answer faces. The loop earns no exemption.
- **Writes:** none. The loop can *read* five ways; the only paths into the index remain the gated flows, untouched.

Given that lattice, swap the model behind the loop for one twice as capable and ask what changes. It chooses better search wordings; it walks the *right* edge instead of a plausible one; it knows when to stop. What *cannot* change: an unsupported answer, an unverified citation, an ungated write — the lattice holds those fixed. **Capability scales; guarantees don't move.** That sentence is the design goal of the entire phase, and this loop is its purest expression. It is also why the plan sends Prompt 39 to the deepest review tier — the lattice is only as good as its implementation, and Module 9 teaches you to inspect every strut.

Anchor: Prompt 39's test list — read it as a verification spec: the spy test (refusal never calls the backend), the budget force-conclude tests, the malformed-action test. Each test pins one strut.

Hands-on: explain this lesson to someone in one minute using only the Mathlib analogy and the phrase "capability scales; guarantees don't move." Teaching it is the test — this argument is the one you will need at funding conversations, and in your own head when a PR makes the loop "just a bit more flexible."

Lesson 5.4 — Study programs: the sweep

> Module 5, Lesson 4 · ~10 min. The one question this answers: **how does one chavrusa loop become a machine that drafts apparatus for a whole sefer — without auto-approving a single thing?**

Answering questions is the agent's retail mode. The wholesale mode: point the same bounded loop at a **sefer, unit by unit, in order** — for each chapter or section, "propose typed relations, term usages, and mashal candidates for this unit, grounded in co-retrieved corpus context, with exact refs." Every proposal runs the existing faithfulness check; survivors land as `status="proposed"` relations and inquiry items. **Nothing is auto-approved.** The sweep's entire output is a *queue for the Rav*, pre-filtered by machine verification and ranked by Module 4's logic. The gate is not relaxed for scale; scale is what the gate was built for.

Engineering-wise, Prompt 40 is deliberately boring — and you should be able to say why that is praise. The pacing, resumability, and crash-survival all reuse the populate drip pattern (a cursor, the injected `Clock`, backoff absorbing 429s) that already survived contact with reality in the bootstrap run. New job engine: none. The one new artifact is the **report** — a POPULATE_REPORT-style funnel per program: drafted → faithfulness-passed → queued, per unit, with an evidence histogram

and spend. That document is how a run is judged, how the funding brief's production run reports itself, and — decisively — how elasticity is measured: run a three-unit sweep with the free drafter, rerun with a top model, same code, and the two funnels *are* the harness-quality number (Module 8).

The sweep is also where the mission's economics become visible. The bottleneck (Lesson 4.1) is the Rav's hour. A sweep converts *model-hours* — cheap today, cheaper every year — into a ranked, verified, typed queue that makes each of the Rav's hours land on the highest-value judgment available. That is what "a harness for future good AI models" means in practice: the better the model, the better the queue, the further one scholar's attention reaches.

Anchor (after Prompt 40): `StudyProgram` + cursor; `maayan study run / status / report`; the no-gate-bypass test (assert *nothing* lands approved).

Hands-on (after Prompt 40): `maayan study run --task relations --units 3` on Tanya with the free drafter, then read the report and skim the proposed edges with `maayan graph list --status proposed`. Triage five of them as if you were the Rav — approve, dismiss, or "needs the sources open." Time yourself. That timing, times the queue length, is Lesson 4.1's arithmetic made personal.

Module 6 — Chazarah: memory that reviews itself

Goal: understand why accumulated knowledge must be consolidated into curated pages and periodically re-reviewed — Wikipedia's loop and Anki's scheduler, behind our gates — Prompt 41.

Lesson 6.1 — Piles don't compound; pages do

> Module 6, Lesson 1 · ~12 min. The one question this answers: **after a year of approvals, what does "the knowledge base knows about ביטול" look like — and what turns forty scattered items into one teachable sugya?**

Fast-forward a successful year: forty approved items touch ביטול — expert connections, a defined term, sweep edges, a shiur segment, two developments. Each is true, cited, attributed. And a learner asking about ביטול gets... whichever fragments retrieval surfaces this time. The knowledge is *present* but not *organized*; it accumulates without ever being *summed*. Encyclopedias solved exactly this: Wikipedia's talk pages hold sprawling contribution and debate, and periodically someone consolidates the state of discussion into the *article* — versioned, superseding, and itself the new baseline for further debate. Contribution and consolidation are different operations, and a knowledge system needs both.

Prompt 41's **sugya page** is the consolidation move, built with the discipline you now expect:

- **Clustering is deterministic** — shared concept plus graph adjacency (connected components over approved edges), no model judgment about what belongs together;
- **Drafting is the grounded unit** — the compose pattern from Course 1's later phases, section by section (sources → connections → open questions), every line cited from the cluster's own chunks, honest gaps allowed;
- **Approval is versioned** — an approved page indexes as `source="sugya"` chunks, retrievable like everything else; approving version N+1 *supersedes* N, whose chunks retract through the existing audited flow. History kept, never overwritten — the system's own printed-text-vs-layers distinction, applied to its own output.

Why this matters beyond tidiness: sugya pages are where **compounding** becomes architecture. A page distills edges and expert chunks; future questions retrieve the page *alongside* its sources; future connections build on the page; the next version of the page distills those. Knowledge citing knowledge citing knowledge — the compounding-rate metric (Module 8) measures exactly whether this loop has caught. And for the learner, the page is the difference between a search engine and a

rebbi: "here is the sugya, ordered, with its sources and its open questions" is what a human teacher produces after years — and what this system is *for*.

Anchor (after Prompt 41): `maayan/consolidate/`; `maayan sugya draft --concept` / `approve` / `list`; the supersede-retract test.

Hands-on: pick the concept you know best. On paper, outline its sugya page from the current corpus: which sources, which connections, and — most valuable — which *open questions*. When Prompt 41 lands, draft the same page mechanically and compare. Where the machine's draft misses what you knew belongs, you have found either a missing edge (add it through the gate) or a clustering blind spot (file it).

Lesson 6.2 — Spaced repetition for a corpus

> Module 6, Lesson 2 · ~10 min. The one question this answers: **why must a knowledge base re-review what it already approved — and how does an interval that multiplies on confirmation spend the Rav's attention optimally?**

Human memory research found something counterintuitive and robust: review is most efficient not on a fixed schedule but on an **expanding** one — recall an item successfully and the optimal next review moves *further* out; fail and it snaps back close. Every serious flashcard system (Anki, SuperMemo) is built on this, because it concentrates scarce review effort exactly where memory is weakest.

Now see the knowledge base as the thing being remembered. Approval was a judgment made at a moment, with the evidence then in view — and judgments age. A bootstrap-era edge was approved when the alternative was nothing; by top-model standards it may be embarrassing. An edge nothing has retrieved in a year is either irrelevant or awaiting its question — a human should glance at which. A page superseded twice signals a live, moving sugya. Approval-forever quietly turns yesterday's floor into tomorrow's ceiling.

Prompt 41's **chazarah scheduler** (the term is exact — chazarah is review as a first-class part of learning, not an afterthought) queues `re_review` inquiry items for: items with weak evidence codes past a config age; approved items never cited in any logged answer (join the graph against the AskLog — Module 4's telemetry paying a second dividend); confidence below threshold. And the Anki move: **each confirmation multiplies the item's next-review interval.** Knowledge the Rav has re-affirmed twice goes quiet for a long time; the queue naturally drains toward the doubtful. Confirmed knowledge costs less and less attention; doubtful knowledge cannot hide. Lesson 4.1's constraint, honored in the small.

House note, so you can review the implementation: no daemon, no cron inside the app — `maayan chazarah` is a *pass you run*, and every interval computation rides the injected `Clock`, which is why

its tests can simulate a year in milliseconds with `FakeClock`. If you find a `time.sleep` or a wall-clock read in that PR, you have found the bug.

Anchor (after Prompt 41): the scheduler in `maayan/consolidate/`; the interval-multiplier config; the FakeClock tests.

Hands-on (after Prompt 41): set `CHAZARAH_BOOTSTRAP_DAYS` low in a scratch `.env`, run `maayan chazarah`, and watch the bootstrap edges surface as re_review items. Confirm one, run the pass again, and verify it does not resurface — then read the stored next-review date and see the interval stretched.

Module 7 — The shape of a ma'amar: structural depth

Goal: understand chunking as a lossy design decision, how parent context restores the flow of a discourse without corrupting citations, and how gated summaries give conceptual questions a place to land — Prompt 42.

Lesson 7.1 — Chunking is a lossy decision

> Module 7, Lesson 1 · ~12 min. The one question this answers: **what did we lose when we cut the seforim into passages — and how do we give the model the flow of the ma'amar back without touching what a citation means?**

Course 1 taught the ingestion rule: preserve each text's natural unit — a chapter's passage, an os, a se'if — as a chunk. It was the right call, and it has a cost that Phase 7 finally pays down. A chassidic ma'amar is not a bag of passages; it is a **discourse** — a question opens, a mashal is built, the nimshal turns it, an earlier kushya resolves. Cutting it into retrieval-sized units is *lossy exactly at the seams*: passage 14 says "וזהו ענין הביטול הנ"ל" — *this is the aforementioned bitul* — and the antecedent lives in passage 11, which retrieval did not return. The model answers from a sentence whose subject it literally cannot see.

The tension to resolve: the **unit of citation** must stay small (a citation's honesty is its precision — "Torah Ohr, Miketz 5:11", checkable in a minute), but the **unit of comprehension** is larger (the model needs the surrounding flow). The industry pattern is parent-document retrieval: *retrieve small, read large.* Prompt 42's version — with the invariant that makes it ours:

- Retrieval runs on passages, exactly as today; hits are found, scored, gated on passages.
- At generation time only, each hit may carry a **window of its neighbors** from the same section (by `section_path` order, up to `PARENT_CONTEXT_CHARS`) into the model's context.
- **The citation remains the exact retrieved passage — never a neighbor.** The window informs; only the hit is cited. Prompt 42 requires a test asserting precisely this, and you should be able to say why it is the lesson's load-bearing line: the moment a citation can point at text retrieval never scored, the citation stops meaning "this passage supports this claim" and the audit trail quietly rots — unsupported-neighbor text influencing answers under a supported passage's name.

Note the family resemblance: like graph expansion (Lesson 3.1), parent context *enriches what the model sees after the gate has ruled* and changes nothing about how the gate rules. By now you should

recognize the shape on sight — every Phase 7 enrichment lives downstream of the gate. All of them. Always.

Anchor (after Prompt 42): the window construction in the RAG prompt builder; `PARENT_CONTEXT_CHARS` (default 0 — off; measurement first, Module 8); the citation-integrity test.

Hands-on: find a real seam today: search for a passage containing הנ"ל or וזהו, and read it alone — then open its full section in the reader (source-in-context, Course 2 era) and read it in place. The difference in *your* comprehension is exactly what the window gives the model.

Lesson 7.2 — Summaries as a gated layer

> Module 7, Lesson 2 · ~10 min. The one question this answers: **where should a question land when its true answer is a whole chapter — and how do we add summaries without diluting the trust core?**

Some questions are passage-sized ("what does this os say about the mashal of the king?"). Others are chapter-sized ("what is Tanya chapter 32 *about*?" — or a conceptual query whose best grounding is a chapter's argument, not any single passage of it). Today the second kind lands awkwardly: retrieval scatters across fragments and the synthesis must reconstruct an arc nobody stored. The research pattern (RAPTOR) is to index *summaries above the leaves* so queries can land at the right altitude. Prompt 42 takes exactly one level of it — chapter/section summaries — and runs it through the machinery this course has made familiar:

- Drafted **grounded-only** from the section's own chunks, by the drip (paced, resumable), faithfulness-checked;
- **Gated** — proposed, human-approved, like develop; a summary is a *derived layer*, and no derived layer enters the index unreviewed;
- Indexed as `source="summary"` chunks carrying parent ref + child list in metadata — so the apparatus and reader can *lead down* from a summary into the exact osiyos;
- **Retractable**, like every layer; printed text untouched, as ever;
- Boosted via `summary_boost` in the same single place all source boosts apply — no special-cased retrieval logic, per the standing rule.

Also in Prompt 42, small but chassidus-shaped: **mashal/nimshal discourse tags** — metadata proposed by the sweeps, approved through the gate, marking which passages are the analogy and which its application. A retrieval axis no general-purpose RAG has, because no general-purpose corpus argues by mashal the way ours does. And by now the recurring question — "does this new layer weaken any guarantee?" — you can answer in one breath: summaries are chunks; chunks face the gate, the threshold, retraction, and provenance like everything else. The trust core doesn't know Prompt 42 happened.

Anchor (after Prompt 42): `maayan summarize --book`; `summary_boost`; the eval run with summaries on/off that the prompt requires be recorded in the phase report.

Hands-on (after Prompt 42): ask "במה עוסק פרק לב בתניא?" with summaries off, then on. With them on, the answer should cite the chapter summary *and* the margins should walk you down to passages. Then check `maayan eval --captured` moved (or didn't) — altitude features must pay in numbers, not vibes.

Module 8 — Measuring the harness

Goal: know the four numbers that define success, understand the benchmark's blinded design and its ablation ladder, and see why the phase ends with a program of judgment work for a top model.

Lesson 8.1 — The four numbers

> Module 8, Lesson 1 · ~12 min. The one question this answers: **if you could see only four charts a quarter from now, which four tell you whether Phase 7 worked?**

"Is the system better?" is not one question; it is four, and each has a number:

1. **Model elasticity** — *does a better model produce more approved knowledge, same code?* Measured by rerunning an identical pipeline with only the drafter swapped. We hold the baseline already: the free-model bootstrap converted 16 of 46 term candidates and 12 of 24 connections. The top-model rerun's funnel against those denominators **is** the harness-quality measurement — a steep gain means the harness was the bottleneck's *remover*; a flat one means the harness itself is the bottleneck, and §7's critique ask exists for exactly that outcome.
2. **Expert leverage** — *approved items per expert-hour.* The binding constraint (Lesson 4.1), priced. The inquiry queue and flywheel exist to move this; gate timestamps already carry the raw data.
3. **Compounding rate** — *what fraction of new approvals ground partly on prior layered knowledge?* The loop-feeding-itself number. The bootstrap produced exactly one such case (אהבה בתענוגים, drafted partly on a prior expert chunk) — call it a proof of concept at n=1. Sugya pages and graph walking should move it; if it stays near zero, we are still a pile with better tooling, and honesty requires saying so.
4. **Benchmark win-rate** — the external check: grounded maayan against a frontier model closed-book, blinded (Lesson 8.2). The number the paper and the raise stand on.

Discipline for reading all four: **decide the direction you expect before you look** (the plan pins benchmark checkpoints after Prompts 35 and 40 for this reason); prefer a number that can *fail* — a metric that only ever flatters is decoration; and when a number disappoints, the finding is the gift: it names the next quarter's work. The bootstrap report's "cross-parsha, not cross-work" finding was exactly that gift, and Prompt 34 is its direct descendant.

Anchor: the metrics table in BUILD_PLAN_PHASE7.md §1; POPULATE_REPORT.md for the standing baselines.

Hands-on: write down, now, your predictions: top-model term conversion (out of 46), and whether A-graph beats A-synth on cross-text recall (Lesson 8.2). Seal them in a file in the repo. Calibration is a skill, and it is only trained by scoring yourself against numbers you committed to in advance.

Lesson 8.2 — The benchmark, blinded

> Module 8, Lesson 2 · ~12 min. The one question this answers: **how do we get a defensible answer to "does grounded beat frontier-from-memory here" — defensible meaning: designed so we cannot fool ourselves?**

The claim at stake is falsifiable and precise: *on chassidus/Kabbalah primary texts, a grounded system over the actual sources beats a frontier model answering from memory — particularly on citation faithfulness and honest refusal.* The design (BENCHMARK_PLAN.md) earns the word "defensible" through four choices, each closing a specific self-deception:

- **An expert gold set, stratified against us.** ~60 questions written by the Rav — including a **Tanya control stratum where the frontier model is strongest** (it has read every English Tanya text on the internet) and negatives where the only correct answer is refusal. A benchmark built only where we expect to win would be marketing.
- **A blinded pairwise judge.** A third model — never either contestant — grades answers presented in randomized order as "Answer 1/Answer 2," identities hidden. LLM judges demonstrably favor themselves and their own style; blinding is the minimum honest countermeasure, and the caveats section of the paper says so plainly.
- **An ablation ladder inside Arm A.** A-text (raw sources only) → A-synth (+ terms and connections) → **A-graph** (+ graph expansion; added by Prompt 35) → **A-agent** (the chavrusa loop; added after Prompt 39). The ladder answers the question a single score cannot: *which investment produced the lift?* Each Phase 7 feature must earn its rung — this is where "measurement first" gets enforced.
- **Refusal correctness as a first-class metric.** For negatives, the *right* answer is "the sources do not address this." A frontier model almost never says that from memory; maayan says it by construction. If the domain's failure mode is confident fabrication of mekoros — and it is — then refusing correctly is not a consolation metric. It is the product.

Note also what the flywheel changed about this picture: the expert's 60 questions are the *exam*, hand-set and rare; the captured goldset (Lesson 4.3) is the *homework*, growing automatically. Rungs that once waited on gold-set authorship now have a second, self-replenishing evaluation surface.

Anchor: `maayan/eval/benchmark.py` (the blinded pairwise judge and closed-book runner exist today); the rung table in BENCHMARK_PLAN.md §7.

Hands-on: write two gold-set questions yourself in the plan's YAML schema — one cross-text, one negative. Writing a *fair* negative (genuinely unanswerable from this corpus, not merely obscure) is harder than it looks; doing it once makes you a competent reviewer of the Rav's sixty.

Lesson 8.3 — After the harness: the top-model program

> Module 8, Lesson 3 · ~10 min. The one question this answers: **when the ten prompts are green, what is left — and why is every remaining task one where judgment, not code, is the input?**

Phase 7's endgame (BUILD_PLAN_PHASE7.md §7) is five asks for a top model, and the selection principle behind them is this course's oldest rule read in reverse. Execution work is delegated to smaller models *because verifiers catch their mistakes*. Whatever remains is work **where no automated verifier exists** — where the output is a judgment, checkable only by another judge. That is precisely where top-model (and Rav) hours belong:

1. **Ontology review, with the Rav** — the relation vocabulary is schema; a wrong verb set deforms every edge poured into it for years. No test can tell you the vocabulary is *wise*.
2. **The Great Sweep** — the full-corpus study program with a top drafter, producing the first machine-drafted, human-gated mareh-mekomos apparatus for chassidus — and, against the bootstrap baselines, the headline elasticity number.
3. **Bikores** — a *different* frontier model, paid to attack every approved item: misattributions, flattened machlokes, meshalim hung on the wrong nimshal. A knowledge base that has survived a funded adversary is a different asset class from one that has merely been built. (Different model than the drafter — correlated errors audit nothing.)
4. **The synthesis exam** — the benchmark's final form, plus one unmetricable test: the top model, inside the harness, composes a new sugya page on a topic the Rav names — and the Rav grades it as he would a talmid's chazarah. Then the paper.
5. **The harness critique** — the recursive ask: the top model works a sustained session inside the harness and reports, as an engineer, where the action space bound it — the tool it reached for that didn't exist, the budget that cut a real line of inquiry. That report *is* the Phase 8 plan. A harness for future models must be iterated **with** future models; this is the loop closing over the harness itself.

Anchor: BUILD_PLAN_PHASE7.md §7.

Hands-on: rank the five by expected value per dollar, in your own judgment, and write one sentence defending your #1. There is a defensible case for at least three different orderings — which is itself the lesson: this, too, is judgment work.

Module 9 — How you will build it

Goal: your job description — the execution protocol that assigns work by model lane, and the reviewer's eye that keeps the trust core intact while smaller models do the typing.

Lesson 9.1 — Which model does what, and why

> Module 9, Lesson 1 · ~10 min. The one question this answers: **by what rule is every piece of Phase 7 work assigned to a top model, a smaller model, or a human — and what does the harness in `.claude/` enforce about it?**

The rule, from the plan's execution protocol (§6), fits in a sentence: **spend the top model where the verifier is weak and the blast radius is high; spend smaller models where the verifier is strong and the spec is already written.** It is the credo's corollary applied to the *development process itself*: Prompts 33–42 each carry their own verifier (requirements, tests, the `make test / typecheck / lint` gate), so executing them is delegable; judgment work — schema, trust-core review, benchmark reading, plan repair — is not.

Three lanes, in practice:

- **Execution (Sonnet-class; Haiku for chores):** one prompt per fresh session, one branch (`prompt33-graph-spine`), the prompt pasted *verbatim* from the plan. The prompt plus CLAUDE.md is the whole contract; a fresh session per prompt prevents drift — accumulated conversational context is how a model talks itself into "improvements" nobody asked for.
- **Judgment (top model):** plan authoring and repair, the Prompt 33 vocabulary session, PR review (Lesson 9.2), benchmark checkpoints, everything in §7. And a standing rule worth its own line: **when execution stalls, fix the prompt, not the code** — bring the failure to a judgment session, amend the plan, rerun. The plan stays the single source of truth and every fix becomes reusable.
- **Runtime (config, separate budget):** the models *inside* maayan — drafters, judges — are an OpenRouter setting, never a dev-session choice. Don't conflate the budgets.

And because a protocol that lives in a doc is a protocol that gets skipped, two pieces are *enforced by the repo itself*: a **Stop hook** (`.claude/hooks/stop-verify.sh`) runs `make lint` and `make typecheck` whenever a session ends with Python files changed — a red branch is pushed back into the session to fix, mechanically, before it can be handed off; and the project permission allowlist pre-approves exactly the read-only verification loop (pytest, ruff check, mypy, the make targets) so an execution session never stalls at a prompt while running the checks we *want* it running constantly.

Small things — and they are the credo again, in miniature: verifiers in the loop, capability where it is safe, friction where it is not.

Anchor: BUILD_PLAN_PHASE7.md §6; `.claude/settings.json` and `.claude/hooks/stop-verify.sh` in this repo.

Hands-on: classify these four tasks by lane: (a) implement Prompt 36's apparatus service; (b) decide whether `is_mashal_for` should be split into mashal/nimshal directions; (c) regenerate the curriculum PDF; (d) read the first A-graph ablation numbers and decide whether Prompt 35's budget default is right. (a: execution. b: judgment — schema, with the Rav. c: chore lane. d: judgment — a number is not a decision.)

Lesson 9.2 — The reviewer's eye

> Module 9, Lesson 2 · ~14 min. The one question this answers: **when a smaller model's PR arrives green — tests passing, types clean — what do you personally look for before it merges?**

Everything this course taught converges here, because the review checklist is nothing but the invariants — and the failures it hunts are precisely the ones that arrive with green tests. A test suite verifies what its author thought to assert; the bugs that matter in this repo are the ones a plausible implementation introduces *around* the assertions. The plan's five checks, each with the concrete smell you are looking for:

1. **Default-deny is not bypassed or weakened.** Smell: any *second* implementation of the gate ("I computed the threshold check locally here...") — one gate, reused, or it is wrong (Lesson 3.2). In graph/agent code, trace the refusal path with your finger and confirm no model call and no store call is reachable from it.
2. **No write path skips its gate or drops provenance.** Smell: a default author (`author: str = "system"` is a bug, not a convenience — author is *required and sticky*, Course 1 law); a store method that flips status to `approved` anywhere outside the one approval flow; a "convenience" CLI flag like `--auto-approve` in a sweep. Nothing is auto-approved. Nothing.
3. **Printed text stays immutable; retraction is integrated, not forked.** Smell: any UPDATE or DELETE touching `sefaria` / `chabad` chunks; a new "remove" method on a store instead of a call into the existing audited retract flow.
4. **Config, not constants; no secrets.** Smell: a magic `0.6` inline ("it worked in my run"), a model name in logic, a key in a log line. Every tunable lands in `config.py` — that is also what makes the elasticity experiment possible at all (swap the drafter in one place).
5. **Tests mock the world; time rides the Clock.** Smell: a test that "just quickly" hits Sefaria; a `time.sleep`; a wall-clock read in scheduler math (Lesson 6.2). If the chazarah tests cannot

simulate a year with `FakeClock`, the implementation is coupled where it must not be.

Method, not just list: review the *diff against the prompt*, not the diff alone — the prompt said what should exist; anything extra is drift to question, anything missing is a gap the green tests didn't cover. For the two trust-core PRs (35 and 39), run the adversarial hands-ons from Lessons 3.2 and 5.3 yourself, live, before approving. And when you find a real problem: the fix flows *into the plan* (Lesson 9.1's standing rule), so the next execution session — and the next project — inherits it.

One closing symmetry, and it is the whole course in a sentence: the process you now run — smaller models drafting inside typed contracts, machine verifiers filtering, scarce human judgment aimed at exactly the items the verifiers cannot decide — **is the system itself.** The Rav gating the graph and you gating the PRs are the same architecture at two scales. Build the one the way you would be judged for building the other.

Anchor: the reviewer checklist in BUILD_PLAN_PHASE7.md §6; CLAUDE.md's house rules — reread them now; after this course they read less like rules and more like the load-bearing walls they are.

Hands-on: take any merged PR from Phases 1–6 (`git log --merges`) and review it against the five checks, as practice on code known to be good — learn what *clean* looks like so deviation becomes visible on sight. Then write your own sixth check: the thing *you* would most plausibly miss. Mine is written into Lesson 3.2. Yours goes in your review template.

Coda — The mission, restated

The printed texts are common property. What is scarce — what this whole project exists to keep — is the reasoning of those who know how to learn them: which passage unlocks which, where the mashal lives, what is a real machlokes and what is two madreigos speaking past each other. Courses 1 and 2 built a system that could *hold* that reasoning faithfully: grounded, cited, refusing, gated. This course taught the phase that lets it **compound**: typed so it composes, evidenced so it is honest about itself, walked so it reaches, questioned so it grows where growth is needed, consolidated so it teaches, measured so we cannot fool ourselves — and harnessed so that every year's better models pour their gains into the same gated, attributed, checkable wellspring instead of washing past it.

A beis medrash with memory is not a metaphor for a database. It is a claim about what happens when one scholar's hour of judgment is banked, structured, and made retrievable for every learner who comes after — עד ביאת גואל צדק.

Glossary — Phase 7 terms

Term	Meaning
Apparatus / margins	The per-ref view of everything asserted about a passage — relations, terms, expert layer — assembled read-only (Prompt 36).
Chazarah scheduler	The re-review pass: aging or weakly-evidenced knowledge resurfaces as inquiry items on an expanding interval (Prompt 41).
Compounding rate	Fraction of new approvals grounded partly on prior layered knowledge — the loop-feeds-itself metric.
Concept	A first-class node for an idea (ביטול, צמצום): canonical name, aliases, kind, optional lexicon link (Prompt 33).
Elasticity	How much approved-knowledge yield improves when only the model improves, same code. The harness-quality metric.
Endpoint	A relation's subject/object: a ref *or* a concept, as a discriminated union — never a bare string.
Evidence code	The permanent tag for *how an assertion is known*: expert / curated / deterministic / derived_approved / derived_bootstrap.
Inquiry queue	The ranked list of machine-generated questions for the Rav: refusals, under-grounded terms, low-evidence items, contradictions (Prompt 37).
Captured goldset	Eval examples auto-drafted from every approval — the regression suite that grows with the knowledge (Prompt 38).
Pasuk join	The deterministic rule: two corpus refs quoting the same verse gain a `parallels` edge — cross-work linkage at $0 (Prompt 34).
Relation	One typed, evidenced, gated edge: subject, verb (`rel_type`), object, evidence, confidence, status, author (Prompt 33).
Study program / sweep	The chavrusa agent run over a sefer unit by unit, drafting proposals for the gates — never auto-approving (Prompt 40).
StudyTrace	The persisted record of an agent run: every action, result digest, and budget spent — a talmid's notebook (Prompt 39).
Sugya page	A versioned, gated consolidation of everything approved about one concept — the article to the contributions' talk page (Prompt 41).
Via	The provenance a graph-expanded retrieval hit carries: which relation, by whom, brought this source to the table (Prompt 35).

> **Regenerating the PDF:** this course is a single self-contained document. Convert any time with the repo's `md-to-pdf` skill on `docs/CURRICULUM3.md`. Like the other curriculum PDFs, the output is regenerated on demand and not committed.